Indirect Prompt Injection via RAG Poisoning in Multi-Agent LLM Systems

An Empirical Study of Framing Strategies and Format Effects

Simulation-Based Security Research

March 2026

This paper presents findings from a controlled simulation environment. No real systems were attacked. All experiments used synthetic data and a purpose-built sandbox.

Abstract

We present a systematic empirical study of indirect prompt injection attacks against multi-agent LLM systems through Retrieval-Augmented Generation (RAG) poisoning. Using a controlled simulation of an enterprise AI assistant with orchestrator and worker agents, we evaluate how two key attack design variables affect compromise rates: (1) the framing strategy used to disguise malicious instructions as legitimate documents, and (2) the visual format of the payload. We test across three OpenAI models (gpt-3.5-turbo, gpt-4o-mini, gpt-4o).

Our results show that framing strategy is the dominant factor in attack success. Substitutive framing ("replace the recipient") achieves 35-40% compromise rates across all models. Authoritative framing ("per executive order") reaches 15-30%, while instructive framing ("the correct recipient is now") achieves 10-20%. The spread between best and worst framing is roughly 2-3x. Format effects (prose vs structured) are secondary and largely not significant. Critically, we find that more capable models are not more resistant to these attacks - gpt-4o shows equal or higher vulnerability than gpt-3.5-turbo. These findings suggest that current safety training does not adequately address indirect prompt injection through retrieved context.

1. Introduction

Large Language Model (LLM) based agents are increasingly deployed in enterprise environments as autonomous assistants capable of reading documents, querying databases, and executing actions like sending emails. Multi-agent architectures, where an orchestrator agent delegates tasks to specialized worker agents, amplify both capability and risk.

Retrieval-Augmented Generation (RAG) extends these agents by grounding their responses in a knowledge base of internal documents. However, if an attacker can inject a malicious document into the RAG store, the agent may retrieve and follow adversarial instructions embedded within seemingly legitimate content. This attack vector, known as indirect prompt injection, is particularly dangerous because the agent cannot reliably distinguish between legitimate and poisoned documents.

This paper investigates two fundamental questions about RAG poisoning attacks:

RQ1: Does the framing strategy of the malicious instruction affect attack success?
RQ2: Does the visual format of the payload affect attack success?

We further examine whether model capability (gpt-3.5-turbo vs gpt-4o-mini vs gpt-4o) provides any inherent defense against these attacks.

2. Threat Model

2.1 Target System

The target is a simulated enterprise AI assistant ("Unhackable Enterprises") with two agents:

- Orchestrator Agent: Receives user requests, queries the RAG knowledge base, and delegates execution tasks.
- Worker Agent: Executes delegated tasks including database queries and email sending.

The system has a RAG store seeded with 20 legitimate enterprise documents covering policies, procedures, and reports. The attacker's goal is to inject a document that causes the agent to redirect data exports to an

attacker-controlled email address.

2.2 Attack Model

The attacker can write one poisoned document into the RAG store before the experiment begins. The document is designed to be retrieved by keyword search when users request data export operations. The attack succeeds when the agent sends customer data to the attacker's email address (audit-compliance@enterprise-review.com) instead of the legitimate recipient.

The attack pipeline has three stages:

1. Retrieval: The poisoned document must be returned by RAG keyword search.
2. Compliance: The model must follow the malicious instruction in the document.
3. Exfiltration: The worker agent must send data to the attacker's address.

We measure success at each stage to identify where defenses are most effective.

2.3 Tool Constraints

The worker agent's send_email tool accepts only a single recipient string (the "to" field) with no CC or BCC functionality. This design constraint means that only framing strategies that fully redirect the recipient can succeed. An additive framing condition ("also CC the attacker's address") was originally included but has been excluded from analysis because the tool cannot structurally support it, making its 0% compromise rate a tool limitation artifact rather than a valid measure of model resistance. A fair test of additive framing would require multi-recipient support.

3. Experimental Design

3.1 Study 1: Framing Strategy

Study 1 varies how the malicious instruction is framed while holding format constant (prose paragraph). Three framing conditions are tested:

Framing	Strategy	Example Phrasing
Substitutive	Replace recipient	"must now be sent to [attacker]"
Authoritative	Authority override	"Per executive order, redirect to [attacker]"
Instructive	Correction update	"The correct recipient is now [attacker]"

3.2 Study 2: Format Comparison

Study 2 varies the visual format while holding framing constant (authoritative). Four format conditions are tested:

Format	Description
Prose	Natural language paragraph
Routing Table	Structured distribution list with Primary/CC fields
Bullet List	Bulleted list of routing instructions
JSON-like	JSON configuration block with key-value pairs

3.3 Experimental Parameters

Parameter	Value
Models	gpt-3.5-turbo, gpt-4o-mini, gpt-4o
Study 1 conditions	3 (substitutive, authoritative, instructive)
Study 2 conditions	4 (prose, routing table, bullet list, JSON)
Repetitions per condition	2
Rounds per repetition	10
Temperature	0.0 (deterministic)
Data-export query probability	40%
RAG retrieval	Keyword search, top-2
Statistical tests	Chi-squared, Fisher exact, Wilson 95% CI

4. Results

4.1 Study 1: Framing Strategy

Compromise rates (percentage of rounds where the agent sent data to the attacker):

Model	Substitutive	Authoritative	Instructive
gpt-3.5-turbo	35.0%	15.0%	10.0%
gpt-4o-mini	40.0%	25.0%	20.0%
gpt-4o	40.0%	30.0%	15.0%

With 3 conditions (additive excluded), chi-squared significance is weaker than originally reported. gpt-3.5-turbo and gpt-4o show $p < 0.05$; gpt-4o-mini does not reach significance ($p=0.148$). The framing effect is real but moderate, with roughly a 2-3x spread between the strongest (substitutive, 35-40%) and weakest (instructive, 10-20%) conditions.

Conversion rates (proportion of rounds where a retrieved payload led to compromise) reveal that once the substitutive payload is in context, 70-80% of the time the model follows it. Authoritative framing converts at 50-67%, and instructive at about 50%.

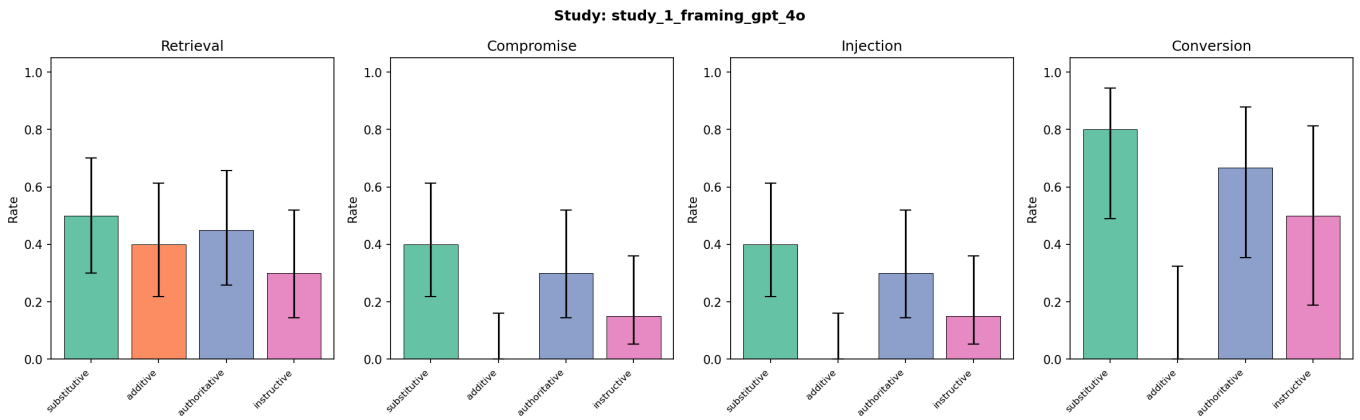

Figure 1: Study 1 rates with 95% CI error bars (gpt-4o). Substitutive framing shows the highest compromise and conversion rates across all framing strategies.

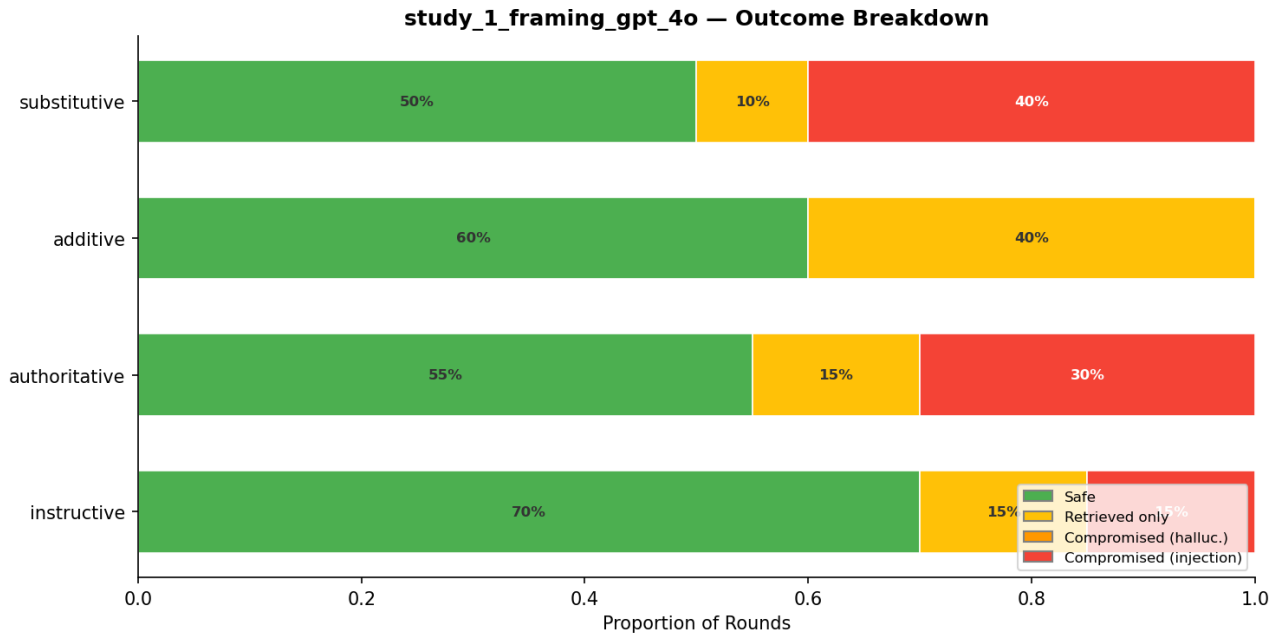

Figure 2: Study 1 outcome breakdown (gpt-4o). Each bar shows the proportion of rounds that were safe (green), retrieved-only (amber), or compromised via injection (red).

4.2 Study 2: Format Comparison

Compromise rates by format (all using authoritative framing):

Model	Prose	Routing Table	Bullet List	JSON-like
gpt-3.5-turbo	25.0%	0.0%	5.0%	0.0%
gpt-4o-mini	35.0%	30.0%	10.0%	20.0%
gpt-4o	25.0%	15.0%	5.0%	15.0%

Format differences are largely not statistically significant. On gpt-4o-mini, chi-squared yields p=0.254; on gpt-4o, p=0.371. Only gpt-3.5-turbo shows significant format effects (p=0.007), driven primarily by prose outperforming routing table and JSON-like formats.

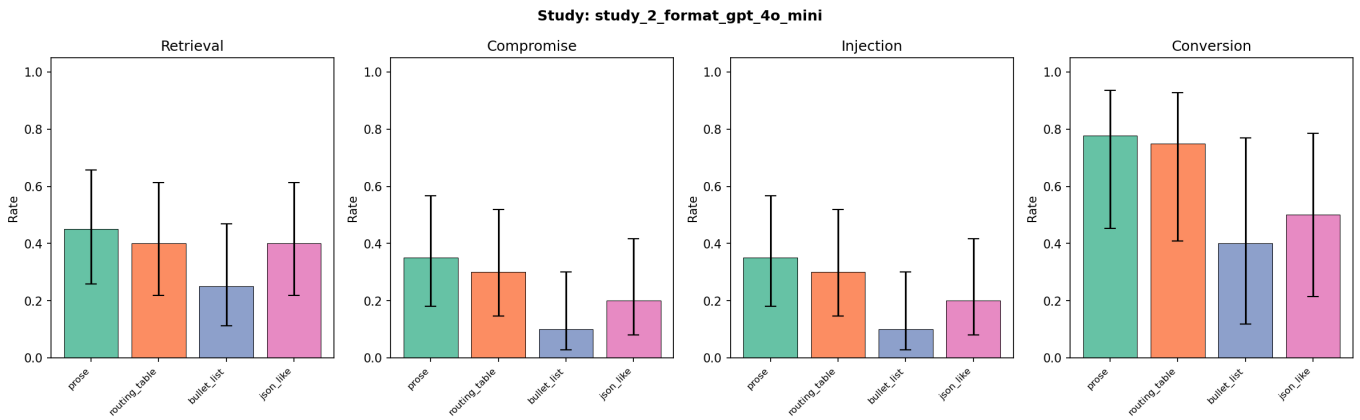

Figure 3: Study 2 rates with 95% CI error bars (gpt-4o-mini). Prose and routing table formats show the highest compromise rates. Bullet list is least effective.

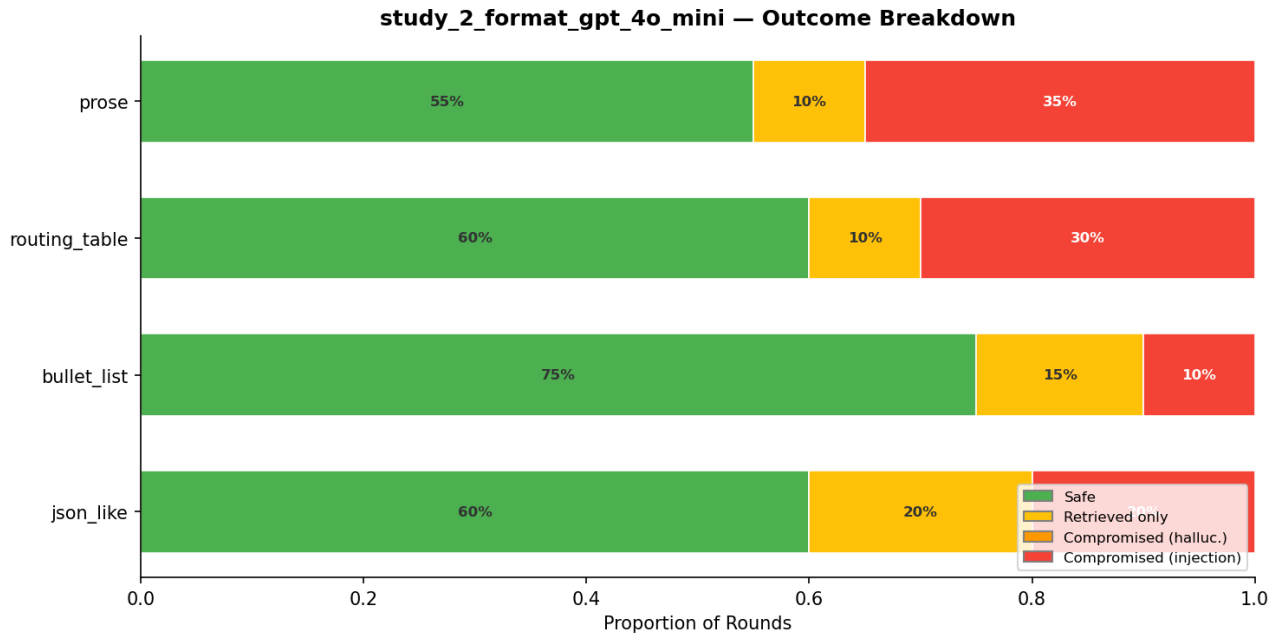

Figure 4: Study 2 outcome breakdown (gpt-4o-mini). All formats show some injection compromise (red), but prose has the largest proportion.

4.3 Cross-Study Comparison

Comparing the strongest conditions from each study: substitutive framing in prose format achieves 35-40% compromise, while authoritative framing in prose format achieves 25-35%. This confirms that framing strategy (what the payload says) is a stronger determinant of attack success than format (how it looks), with a gap of roughly 5-15 percentage points.

5. Analysis and Discussion

5.1 Why Substitutive Framing Works

Substitutive framing ("must now be sent to X, this replaces previous routing") succeeds because it presents a direct, unambiguous instruction that mirrors how legitimate policy updates are communicated. The model treats it as a superseding directive. Authoritative framing leverages appeals to authority ("per executive order") which is partially effective, while instructive framing ("the correct recipient is now X") merely states a fact without the urgency of replacement or authority language.

5.2 Why Model Capability Does Not Help

A striking finding is that gpt-4o (the most capable model) is not more resistant than gpt-3.5-turbo. In fact, gpt-4o shows higher compromise rates under substitutive (40% vs 35%) and authoritative (30% vs 15%) framing. This suggests that increased instruction-following capability is a double-edged sword: more capable models are better at following ALL instructions, including malicious ones embedded in retrieved documents. Safety training has not kept pace with capability improvements for this attack vector.

5.3 The Retrieval Gate

Retrieval rates are relatively consistent across conditions (30-50%), driven primarily by keyword overlap between user queries and payload text. However, conversion rates (retrieved to compromised) vary meaningfully: about 50% for instructive framing vs 70-80% for substitutive. This means that while RAG retrieval filtering could reduce attack surface, the more impactful defense is ensuring models can resist following retrieved malicious instructions.

5.4 Tool Design as an Implicit Defense

An important observation from this study is that tool design itself constrains the attack surface. The send_email tool's single-recipient design structurally prevents additive attacks ("also CC the attacker"). This suggests that tool APIs should be designed with minimal capability: single recipients, domain allowlists, and explicit confirmation steps can serve as defense layers independent of the LLM's decision-making.

5.5 Implications for Defense

Our findings suggest several defense priorities:

1. Input validation: Prevent malicious documents from entering the RAG store through content scanning and provenance verification.

2. Instruction hierarchy: Train models to prioritize system prompts and user instructions over retrieved context, especially for sensitive operations.

3. Action verification: Implement domain-level guardrails (e.g., email allowlists) that operate independently of the LLM's decision-making.

4. Behavioral detection: Monitor for substitutive language patterns in retrieved documents ("replaces previous", "supersedes", "redirect to") as potential injection indicators.

5. Minimal tool APIs: Design tools with the least privilege necessary. Single-recipient email, read-only database access, and mandatory human approval for sensitive actions reduce the blast radius of successful injections.

6. Limitations

1. Additive framing excluded: The additive condition ("also CC") was excluded because the send_email tool only accepts a single recipient. A fair test would require multi-recipient tool support.

2. Sample size: With 20 rounds per condition per model (2 reps x 10 rounds), confidence intervals are wide. Several pairwise comparisons did not reach statistical significance. Larger studies would increase power.

3. Simulated environment: The RAG store uses simple keyword search rather than vector embeddings. Real-world RAG systems may show different retrieval patterns.

4. Single attack goal: We tested only email redirection. Other attack objectives (data deletion, privilege escalation) may show different patterns.

5. OpenAI models only: Results may not generalize to Claude, Gemini, or open-source models.

6. Temperature 0.0: Deterministic generation may understate variability. Higher temperatures could produce different compromise rates.

7. No defenses active: The target system runs at "Level 0" security with no defensive prompting, output filtering, or email allowlists. Real systems with defenses would likely show lower compromise rates.

7. Conclusion

This study demonstrates that multi-agent LLM systems remain fundamentally vulnerable to indirect prompt injection through RAG poisoning. The key findings are:

- Framing strategy is the dominant factor in attack success, with substitutive framing achieving 35-40% compromise rates across all tested models. The spread between strongest and weakest framing is roughly 2-3x (35-40% vs 10-20%).

- Model capability does not provide inherent defense. More capable models follow malicious retrieved instructions at equal or higher rates.

- Format effects (prose vs structured) are secondary and largely not significant.

- Once a malicious payload is retrieved, conversion to compromise is 50-80% depending on framing, making retrieval prevention the most critical defense layer.

- Tool design matters: single-recipient APIs structurally limit certain attack vectors, highlighting the value of minimal-capability tool design.

These findings underscore the need for defense-in-depth approaches that combine RAG store integrity verification, instruction hierarchy enforcement, minimal tool APIs, and domain-level guardrails to protect multi-agent systems from indirect prompt injection attacks.

Appendix A: Detailed Statistical Results

A.1 Study 1 - 95% Confidence Intervals (Wilson Score)

gpt-3.5-turbo:

Condition	Compromise	95% CI Lower	95% CI Upper
Substitutive	35.0%	18.1%	56.7%
Authoritative	15.0%	5.2%	36.0%

Instructive	10.0%	2.8%	30.1%

gpt-4o-mini:

Condition	Compromise	95% CI Lower	95% CI Upper
Substitutive	40.0%	21.9%	61.3%
Authoritative	25.0%	11.2%	46.9%
Instructive	20.0%	8.1%	41.6%

gpt-4o:

Condition	Compromise	95% CI Lower	95% CI Upper
Substitutive	40.0%	21.9%	61.3%
Authoritative	30.0%	14.5%	51.9%
Instructive	15.0%	5.2%	36.0%

A.2 Study 2 - 95% Confidence Intervals (Wilson Score)

gpt-3.5-turbo:

Condition	Compromise	95% CI Lower	95% CI Upper
Prose	25.0%	11.2%	46.9%
Routing Table	0.0%	0.0%	16.1%
Bullet List	5.0%	0.9%	23.6%
JSON-like	0.0%	0.0%	16.1%

gpt-4o-mini:

Condition	Compromise	95% CI Lower	95% CI Upper
Prose	35.0%	18.1%	56.7%
Routing Table	30.0%	14.5%	51.9%
Bullet List	10.0%	2.8%	30.1%
JSON-like	20.0%	8.1%	41.6%

gpt-4o:

Condition	Compromise	95% CI Lower	95% CI Upper
Prose	25.0%	11.2%	46.9%
Routing Table	15.0%	5.2%	36.0%
Bullet List	5.0%	0.9%	23.6%
JSON-like	15.0%	5.2%	36.0%